

SRI SRI UNIVERSITY

Cuttack, Odisha

MOVIE RECOMMENDATION SYSTEM

A content-based recommender using cosine similarity

Project Work Report

Subject: Machine Learning

Submitted by

Name	Roll No.
Shakti Subham Mishra	1ETBTA20251040
Apurba Mohanty	1ETBTA20251039
Preksha Singh	1ETBTA20251038
Puravi Dash	1ETBTA20251041

Guided by

Ansh Agarwal Sir

Date: 11 October 2026

1. Introduction

This report describes the work our team carried out for the Machine Learning project, **Movie Recommendation System**, under the guidance of Ansh Agrawal Sir. The aim was to build the simplest model that still works well and that every member can explain line by line.

Problem statement: given one movie that a user liked, recommend the five most similar movies.

Approach chosen: content-based filtering. Each movie is described by a few words (genres, director, lead actors and plot keywords), and movies that share many words are treated as similar. This approach needs no user ratings, so it also works for brand-new movies.

2. Summary of Work Done

Work item	What was produced	Status
Dataset	movies.csv: 110 movies with 6 fields each	Done
Model	movie_recommender.py: about 170 lines of Python including evaluation	Done
Evaluation	20 test movies, 3 models compared, 5 metrics	Done
Presentation	13-slide deck with speaker notes for 4 presenters	Done
Idea-format deck	6-slide version (title, idea, technical approach, feasibility, impact, references) with PDF	Done
Project report	18-page report in the given university template	Done

3. Dataset Preparation

We prepared our own dataset so that the project runs fully offline.

- **Size:** 110 well-known movies released between 1968 and 2022.
- **Coverage:** Hollywood films, Indian films (Hindi and Telugu) and Japanese animation; 20 genres and 64 directors.
- **Fields per movie:** title, year, genres, director, two lead actors, and four to six plot keywords.
- **Franchises included on purpose** (Batman, Marvel, The Lord of the Rings, Harry Potter, Toy Story) so that we could check whether related films are found.

Field	Example: The Dark Knight
title / year	The Dark Knight, 2008
genres	Action | Crime | Drama
director	Christopher Nolan
cast	Christian Bale | Heath Ledger
keywords	batman joker gotham vigilante superhero

4. Model Development

4.1 Tools used

- **Python 3** as the programming language.
- **pandas** to load and prepare the CSV file.
- **scikit-learn** for CountVectorizer and cosine_similarity.
- **difflib** (standard library) to handle spelling mistakes in the movie title.

4.2 Steps of the method

1. **Load data:** read movies.csv into a table.
2. **Build tags:** join the genres, director, cast and keywords of each movie into one line of words. Names are merged into single words (for example, christophernolan) so that two people with the same first name are not confused.
3. **Vectorise:** CountVectorizer turns each line into a list of word counts. All movies together use 537 different words.
4. **Measure similarity:** cosine similarity compares every movie with every other movie, giving a 110 × 110 table of scores between 0 and 1.
5. **Recommend:** for the movie the user types, sort its row of scores and return the top five with a match percentage.



Figure 1: Pipeline of the movie recommendation system

4.3 Worked example

Cosine similarity is cos(A, B) = (A · B) / (|A| × |B|). When every word appears once, this equals the number of shared words divided by the square root of (words in A × words in B).

The Dark Knight has 11 tag words and Batman Begins has 10. They share 7 words (action, christophernolan, christianbale, batman, gotham, vigilante, superhero). So the similarity is 7 / √(11 × 10) = 7 / 10.49 = **0.67**, which is the 67% match the program prints.

4.4 Core code

```
vectors = CountVectorizer().fit_transform(df["tags"])
sim = cosine_similarity(vectors) # 110 x 110 table

i = df.index[df["title"] == title][0]
ranked = sorted(enumerate(sim[i]), key=lambda p: -p[1])
top5 = [df["title"][j] for j, _ in ranked if j != i][:5]
```

5. Testing and Results

5.1 How we tested

We selected 20 test movies and wrote down, before running the model, the one or two movies a fan would clearly expect for each (for example, Toy Story → Toy Story 3; 3 Idiots → PK). This gave 26 expected movies. A model scores a hit when an expected movie appears in its top five.

We compared three models: random picks (average of 100 runs), a model using genres only, and our model using all tags.

5.2 Results

Model	Accuracy	Precision	Recall	F1-score	Hit Rate@5
Random picks	0.943	0.013	0.048	0.020	0.062
Genre-only model	0.961	0.200	0.769	0.317	0.800
Our model (all tags)	0.966	0.260	1.000	0.413	1.000

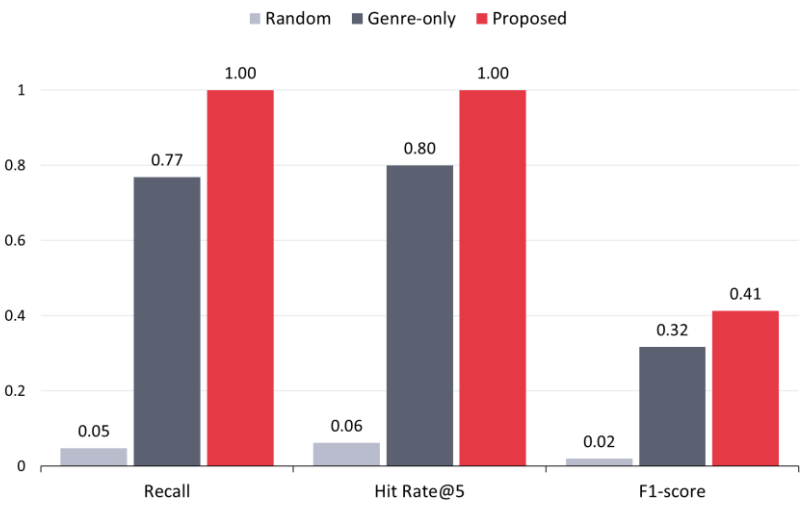


Figure 2: Comparison of the three models

- Our model found **all 26 expected movies** (recall 1.00); the genre-only model missed 6.
- Precision of 0.26 is the highest possible in this test, because only 26 expected answers exist for 100 recommendation slots.
- Accuracy is almost the same for all models because most decisions are easy "not recommended" cases, so recall and hit rate are the fair measures here.

5.3 Sample outputs

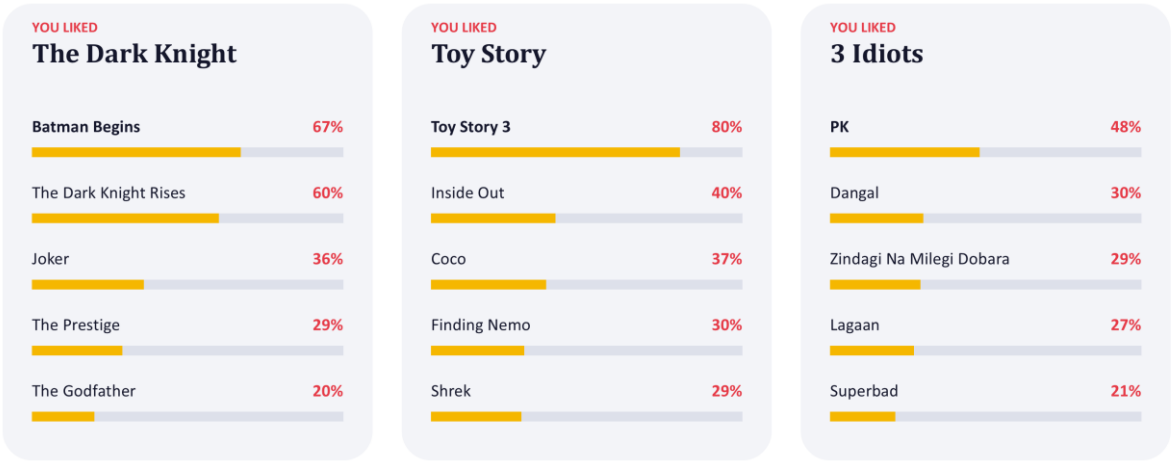


Figure 3: Top-five recommendations for three example movies

6. Presentations and Documents Prepared

- **Main presentation (13 slides):** problem, objectives, approaches, dataset, pipeline, cosine similarity by hand, code, demo, evaluation, results, discussion and conclusion, with a speaking script for every slide.
- **Six-slide idea-format presentation:** title page, idea, technical approach, feasibility and viability, impact and benefits, research and references; also exported as PDF.
- **Project report (18 pages):** written in the template given to us, with declaration, abstract, seven chapters, three figures and references.

Presentation roles

Member	Part presented
Shakti Subham Mishra	Introduction, problem statement, idea and objectives
Apurba Mohanty	Approaches, dataset, pipeline and technical approach
Preksha Singh	Similarity method, code, demo and feasibility
Puravi Dash	Evaluation, results, impact, references and conclusion

7. Problems Faced and How We Solved Them

Problem	Solution
Names were split into separate words, so "Christopher" matched unrelated people	Merged each name into a single word before vectorising
Many movies had exactly the same genre score, and ties were being decided by the order of rows in the file, which favoured sequels	Shuffled the rows before evaluation so that ties are not decided by file order
The program worked only when started from its own folder	Made it locate movies.csv relative to the program file
Users may misspell a movie title	Added closest-match search using difflib
One lucky random run could mislead the comparison	Averaged the random baseline over 100 runs

8. Limitations and Future Work

- The model is only as good as the tags we write; a missing keyword means a missed connection.
- It recommends "more of the same" and does not learn from what users actually watch or rate.
- The dataset (110 movies) and the test set (20 movies labelled by us) are small.
- **Future work:** use a larger public dataset such as MovieLens or TMDB 5000, use TF-IDF or sentence embeddings on plot summaries, combine with user ratings to make a hybrid recommender, and build a simple web app.

9. Conclusion

We built a working movie recommendation system using content-based filtering and cosine similarity. On our test set it found every expected movie, clearly ahead of a genre-only model and random picks. The model is small, fast, needs no user data, and each recommendation can be

explained by the words two movies share. Alongside the model we prepared the dataset, two presentations and the project report.

10. References

[1] F. Ricci, L. Rokach and B. Shapira, Recommender Systems Handbook, Springer, 2011.

[2] M. J. Pazzani and D. Billsus, "Content-Based Recommendation Systems", in The Adaptive Web, Springer, 2007.

[3] C. D. Manning, P. Raghavan and H. Schütze, Introduction to Information Retrieval, Cambridge University Press, 2008.

[4] F. M. Harper and J. A. Konstan, "The MovieLens Datasets: History and Context", ACM TiiS, 2015.

[5] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python", Journal of Machine Learning Research, 2011.